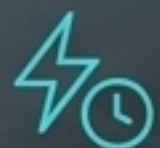


Da Teoria à Prática: A Anatomia de um Projeto Q1

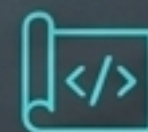


O DNA do Quadrante 1 (Q1)



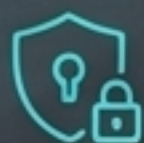
Feedback Rápido

Execução em segundos. Falhas são corrigidas no momento em que o código é digitado.



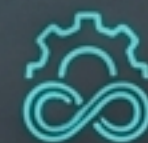
Guiam o Design (TDD)

Testes são escritos antes. Eles definem os contratos e o comportamento esperado das classes.



Prevenção de Regressão

Uma base de segurança absoluta que permite a refatoração sem medo de quebrar o sistema.



Automação Total

Sem intervenção humana. A validação ocorre a cada alteração no repositório.



"A medição da qualidade interna do código." — Baseado no conceito de Kent Beck

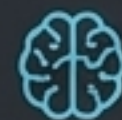
Arquitetura do Projeto PHP (CRUD)



src/Product.php



Módulo de Negócio (O Cérebro)



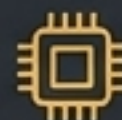
Entidade de domínio. Encapsula validações de limite e lógica de negócio (applyDiscount, sell).



src/ProductRepository.php



Módulo de Persistência (A Memória)



Padrão Repository em memória. Abstrai o acesso a dados e isola o estado do banco.



tests/ ✓

ProductTest.php

ProductRepositoryTest.php



Módulo de Validação (O Escudo)



ProductTest.php e ProductRepositoryTest.php.
A camada que garante os contratos do Q1.

TDD na Prática: Testes Guiando o Design

O Teste

```
testTrimsNameWhitespace() {  
    setter.name(test());  
}
```

O Código Resultante

O setter é obrigado a usar trim() no nome antes de salvar.

O Teste

```
testDiscountRoundsToTwoDecimals  
}
```

O Código Resultante

Uso obrigatório de round() para proteger os valores monetários de erros de ponto flutuante.

O Teste

```
testThrowsExceptionWhenSellingMoreThanStock  
}
```

O Código Resultante

Lança UnderflowException (SPL).
Define semanticamente que a pilha de estoque não pode ficar abaixo de zero.

Matriz Diagnóstica: Unitário vs. Componente



Teste Unitário

ProductTest.php

Escopo

Valida a lógica interna de um único objeto. Isolamento total de dependências.

Exemplo Prático

Verificação de limites de entrada (Boundary Testing), como preço zero ou nomes com 100 caracteres.



Teste de Componente

ProductRepositoryTest.php

Escopo

Valida a interação de um grupo de classes fornecendo um serviço (Produto + Repositório).

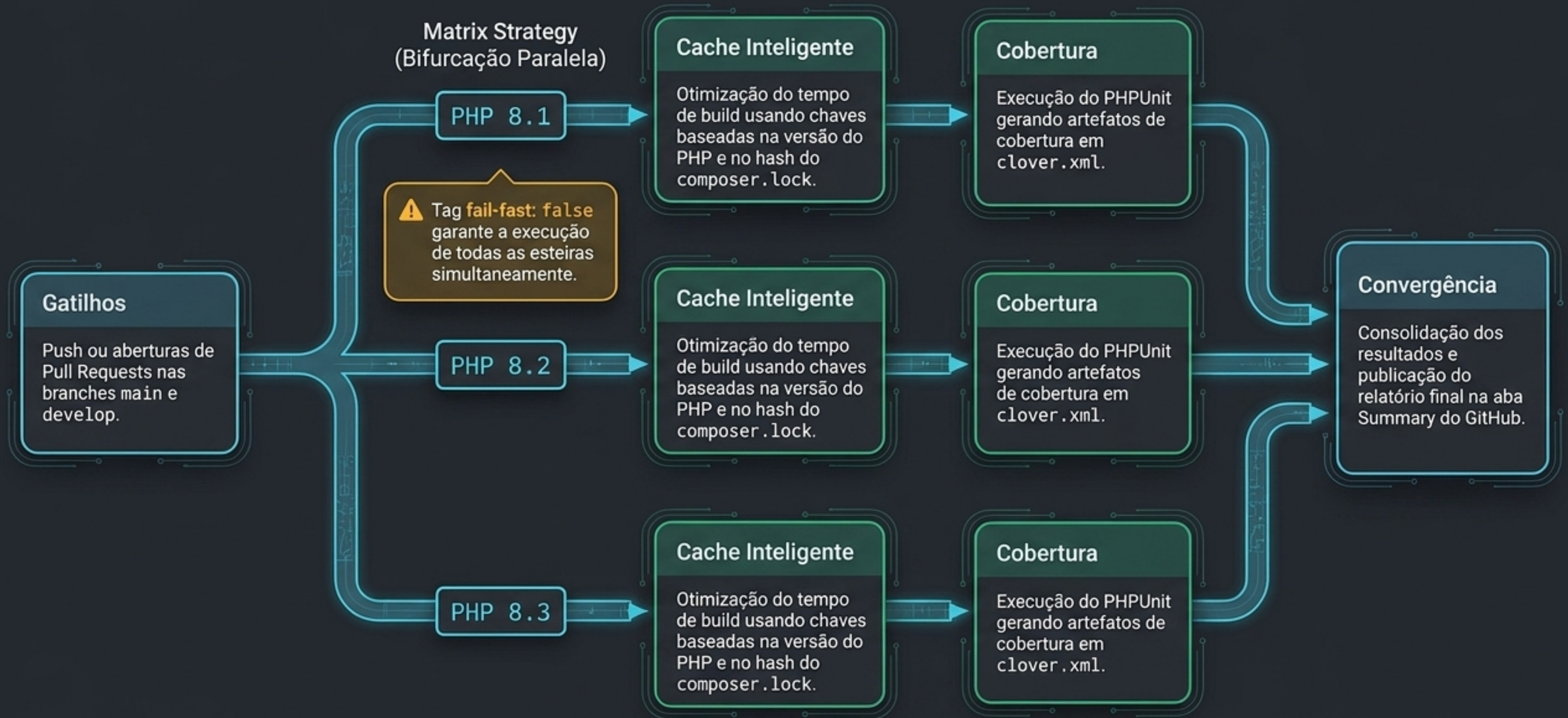
Exemplo Prático

Cenário de Ciclo Completo (Create, Read, Update, Delete).

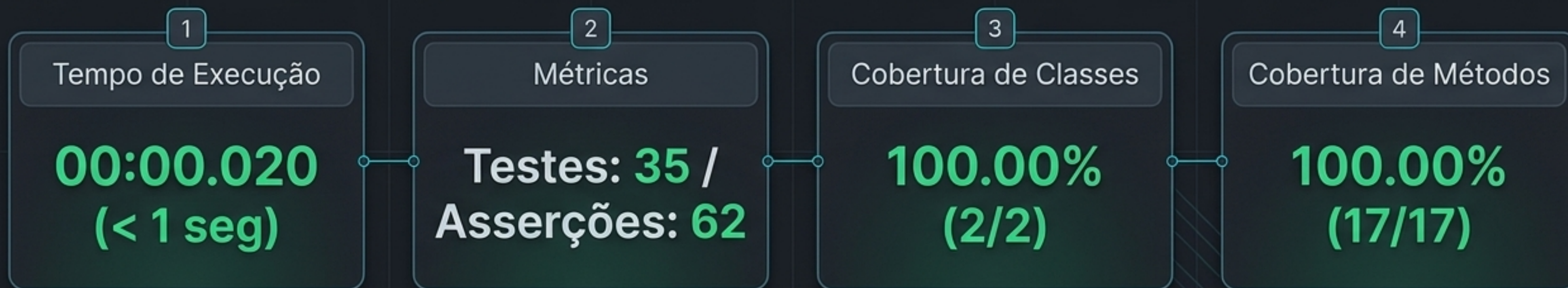
O Princípio do Estado Limpo

O método **setUp()** zera e isola a base de dados em memória antes de cada teste de componente para evitar interferência mútua.

O Motor de Automação: Pipeline CI/CD



O que significa 100% de Cobertura?



⚠ Aviso Crítico sobre a Cobertura:

100% de cobertura é um indicador técnico, não uma garantia comercial. Significa apenas que o interpretador executou cada linha de código ao menos uma vez durante os testes. Não atesta, por si só, a ausência de falhas na lógica humana.

A Fronteira do Quadrante 1

O que o Q1 garantiu



Design foi forjado pelos testes de limite e exceções.



Construção automatizada (CI/CD) validando a segurança do projeto a cada commit.



Contratos tecnológicos e qualidade interna blindados.

O que o Q1 delega para o futuro



Expectativas do Cliente (Q2/Q3): Testes voltados para os negócios (ex: BDD com Behat/Cucumber) ficam fora desta arquitetura.



Requisitos Tecnológicos Macro (Q4): Testes de carga e segurança atuarão na periferia do repositório, não na camada de domínio estruturada aqui.